

AI – TECH

SKRYPT DO LABORATORIUM

UCZENIE MASZYNOWE

LABORATORIUM 5:

Sztuczne sieci neuronowe

dr Tomasz Neumann



**Rzeczpospolita
Polska**

Unia Europejska
Europejski Fundusz
Rozwoju Regionalnego



1. Opis ćwiczenia

Wymagania wstępne:

Osiągnięcie celów ćwiczenia będzie możliwe poprzez właściwe przygotowanie się studenta do zajęć.

Będzie to możliwe poprzez:

- powtórzenie wiedzy nabytej w czasie wykładów,
- zapoznanie się (przed ćwiczeniem) z instrukcją do ćwiczenia ilustrującą podstawowe zagadnienia z zakresu tematyki ćwiczenia.

Ponadto wymagana jest od studenta:

- podstawowa umiejętność programowania,
- podstawowa znajomość języka Python,
- podstawowa znajomość analizy danych,
- umiejętność twórczego myślenia.

Wyżej wymienione wymagania bazują na wiedzy i umiejętności jakie studenci powinni zdobyć w czasie edukacji na I poziomie studiów jak i w ramach przedmiotu „Języki programowania dla sztucznej inteligencji”. W przeciwnym przypadku studenci powinni nadrobić samodzielnie stosowny materiał.

Stanowisko laboratoryjne powinno być wyposażone w komputer z dostępem do sieci komputerowej oraz z następującymi zasobami:

- przykładowe programy (kody programów) podane w załączniku,
- zainstalowany interpreter języka Python w wersji co najmniej 3.6 z możliwością doinstalowania bibliotek standardowych za pomocą pip lub anaconda (najlepiej w ramach dedykowanego środowiska wirtualnego),
- zainstalowany Jupyter Notebook,
- dostęp do rozwiązywania chmurowego Google Colab,
- inne: przeglądarka WWW.

Na komputerze dostępne powinny być zainstalowana instrukcja do ćwiczenia w wersji elektronicznej oraz pliki testowe. Przed zajęciami stacjonarnymi, prowadzący powinien sprawdzić czy dostępne są na stanowisku wymagane zasoby i programy.

Cele ćwiczenia:

Celem ćwiczenia jest praktyczna demonstracja wiedzy teoretycznej, przedstawionej w ramach wykładów. Realizując ćwiczenie laboratoryjne studenci zdobędą szereg nowych umiejętności w zakresie klasyfikacji wieloklasowej, wieloetykietowej oraz wielowyjściowej jak i nabędzie umiejętność implementacji prostych sieci neuronowych bazujących na interfejsie Keras i Tensorflow.

Spodziewane efekty kształcenia - umiejętności i kompetencje:

Student po zajęciach laboratoryjnych będzie posiadał zwiększoną umiejętność przygotowania danych wejściowych do zagadnień klasyfikacji wieloklasowych używając pakietu scikit-learn oraz Keras API wraz z Tensorflow. W trakcie zajęć student zapozna się także z metodami oceny trenowanych modeli oraz będzie w stanie w sposób krytyczny zinterpretować uzyskane wyniki w standardowych problemach klasyfikacji wieloklasowych.

Metody dydaktyczne:

Student w pierwszej kolejności realizuje zadania przykładowe podczas których przygotowuje odpowiednie środowisko programistyczne, poprzez które testuje poszczególne elementy pracy z danymi do klasyfikacji wieloklasowej w pakietach scikit-learn oraz Keras API. Następnie, na podstawie posiadanej wiedzy i zdobytej umiejętności z zadań demonstracyjnych, realizuje przydzielone zadania. Wszystkie kody źródłowe programów wpisuje w dokumencie elektronicznym (lub pokazuje prowadzącemu), który jednocześnie staje się sprawozdaniem z ćwiczenia laboratoryjnego.

Materiały wprowadzające i pomocnicze:

- Python – dokumentacja języka <https://docs.python.org/pl/3/>
- SciKit-Learn – dokumentacja do podstawowej biblioteki uczenia maszynowego stosowanej w języku Python: <https://scikit-learn.org/stable/tutorial/basic/tutorial.html>
- Keras – API przeznaczone dla ludzi i maszyn, oferujący spójne i proste interfejsy, minimalizując liczbę działań użytkownika wymaganych w typowych przypadkach, prezentując jednocześnie

jasne i przydatne komunikaty o błędach. Zawiera także obszerną dokumentację, przewodniki i przykłady dla programistów - <https://keras.io/>

- Tensorflow – to kompleksowa platforma open source do uczenia maszynowego, która ułatwia początkującym jak i ekspertom tworzenie prostych oraz zaawansowanych modeli uczenia maszynowego -<https://www.tensorflow.org/>
- Jupiter notebook – narzędzie ułatwiające implementację kodu, uruchamianie oraz testowanie: <https://realpython.com/jupyter-notebook-introduction/>
- Google Colab – narzędzie umożliwiające pracę z kodem w notebooku Jupiter oraz jego uruchamianie w chmurze (konieczność posiadania konta Google): <https://colab.research.google.com/github/cs231n/cs231n.github.io/blob/master/python-colab.ipynb>

Zasady oceniania/warunek zaliczenia ćwiczenia

Ocenie podlegać będzie realizacja poszczególnych zadań (fragmenty kodu, analizy i wnioski) w ramach danego ćwiczenia laboratoryjnego. Dodatkowo w czasie realizacji ćwiczenia przydzielone zostaną studentowi dodatkowe zadania. Poszczególnym zadaniom przydzielono liczbę punktów jakie można zdobyć za ich wykonanie. Maksymalna liczba punktów wynosi 10.

Wykaz literatury podstawowej do ćwiczenia:

1.	Materiały z wykładu do przedmiotu „Języki programowania dla sztucznej inteligencji”
2.	Materiały z wykładu do przedmiotu „Uczenie maszynowe”
3.	Marcin Szeliga, Praktyczne uczenie maszynowe, PWN 2019
4.	M. Gagolewski, M. Bartoszek, A. Cena, Przetwarzanie i analiza danych w języku Python, PWN, 2016
5.	Sebastian Raschka, Python. Uczenie maszynowe, Helion 2017.
6.	Aurelien Geron, Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow, 2019

2. Przebieg ćwiczenia

L.p.	Zadanie
1.	Zapoznanie się z instrukcją laboratoryjną (przed ćwiczeniem)
2.	Wprowadzenie kierownika ćwiczenia. Zapoznanie się uczestników ze stosowanymi programami, przegląd dokumentacji (5 min.)
3.	Klasyfikacja wieloklasowa, wieloetykietowa i wielowyjściowa w Scikit-Learn (40 min.)
4.	Klasyfikacja przy użyciu Keras i Tensorflow – autoenkoder (25min)
5.	Klasyfikacja przy użyciu Keras na zbiorze CIFAR-10 za pomocą prostej sieci neuronowej (60 min)
6.	Przesłanie/pokazanie zbioru dokumentów/programów do oceny (5 min.)

Uwagi

PRZED przystąpieniem do ćwiczenia należy utworzyć własny, unikalny katalog w lokalizacji wskazanej przez prowadzącego, a następnie należy tam umieścić pliki testowe oraz własne implementacje kodu programów dokonujących rozwiązanie zadanie. Należy pracować wyłącznie na wykonanej kopii plików źródłowych.

3. Wprowadzenie do ćwiczenia

Klasyfikatory wieloklasowe (wielomianowe) służą do rozróżniania większej liczby klas. Niektóre algorytmy (takie jak klasyfikatory SGD, klasyfikatory losowego lasu lub naiwne klasyfikatory bayesowskie) są w stanie natywnie zajmować się wieloma klasami naraz. W przypadku wektorów nośnych czy klasyfikatorów regresji logistycznej mają charakter typowo binarny. Istnieje jednak wiele strategii, dzięki którym możemy przeprowadzać klasyfikację wieloklasową przy użyciu wielu klasyfikatorów binarnych.

Jednym ze sposobów stworzenia systemu klasyfikującego 10 klas obrazów cyfr (od 0 do 9) polega na wyuczeniu 10 klasyfikatorów binarnych, po jednym na każdą cyfrę. Następnie na etapie klasyfikacji obrazu uzyskujemy wynik z każdego klasyfikatora dla danej cyfry i wybieramy klasę, która uzyskała najwyższy wynik. Jest to tzw. strategia jeden przeciw reszcie (one-versus-rest — OvR lub one-versus-all

— OvA). Jeszcze innym rozwiązaniem jest wyuczenie klasyfikatora binarnego dla każdej pary cyfr: jeden rozpoznaje zera i jedynki, drugi — zera i dwójki, jeszcze inny — jedynki i dwójki itd. Jest to strategia jeden przeciw jednemu (one-versus-one — OvO). Jeśli mamy do czynienia z N klasami, należy wytrenować $N \times (N-1)/2$ klasyfikatorów. W przypadku zestawu danych MNIST oznacza to wyuczenie 45 klasyfikatorów binarnych.

Zaletą strategii OvO jest fakt, że każdy klasyfikator musi zostać wytrenowany jedynie wobec części zbioru uczącego składającego się z obydwu porównywanych klas. Pewne algorytmy nie skalują się zbyt dobrze do rozmiarów zbioru uczącego (np. maszyna wektorów nośnych). W takim przypadku preferowane jest stosowanie strategii OvO, ponieważ uczenie wielu klasyfikatorów wobec niewielkich zbiorów danych przebiega szybciej niż trenowanie kilku klasyfikatorów wobec użytych zbiorów przykładów. Jednakże w przypadku większości klasyfikatorów binarnych zalecane jest korzystanie ze strategii OvR.

Moduł `scikit-learn` pozwala zastosować strategię OvO lub OvR, poprzez wykorzystanie klas `OneVsOneClassifier` lub `OneVsRestClassifier`. Wystarczy stworzyć ich wystąpienie i przekazać konstruktorowi klasyfikator (nie musi być on nawet binarny). Poniższy kod przykładu stworzy klasyfikator wieloklasowy (na podstawie klasyfikatora SVC) przy użyciu strategii OvR, co zostało przedstawione w notatniku *Lab5_ex01.ipynb*. W dalszej części pokazane jest także wykorzystanie klasyfikatora SGD, mogącego bezpośrednio klasyfikować przykłady w sposób wieloklasowy stąd też nie ma potrzeby wdrażania powyżej opisanych strategii. Jednak te zaprezentowane przykłady analizowały jedynie pojedyncze dane, przydzielając je wyłącznie do jednej klasy. Istnieją sytuacje, aby klasyfikator wyznaczał wiele klas dla jednego wystąpienia, co zostało przedstawione na przykładzie wykorzystania klasyfikatora *KNeighborsClassifier*. W tym przykładzie została także pokazana uogólniona klasyfikacja wieloetykietowa tj. klasyfikacja wielowyjściowa, pokazująca usuwanie wprowadzonego sztucznie szumu do obrazów cyfr z bazy danych MNIST.

W notatniku drugim została zawarta przykładowa implementacja modelu przy zastosowaniu zaawansowanego API do implementacji autokodera. Autokodery to modele sieci neuronowej złożone z

dwóch bloków, kodera i dekodera, które przetwarzając dane wejściowe x , w celu ich rekonstrukcji na wyjściu x' . Związany jest z tym również termin "auto".

W klasycznym ujęciu autokodery działają na danych wejściowych x bez wyróżniania etykiet y . Rozpatrywany problem polega zatem na nauczaniu modelu takich wag, aby uzyskać idealną rekonstrukcję. Ponieważ nie rozważamy wykorzystania etykiet czy innej formy nadzoru dlatego klasyczne autokodery zaliczane są do grupy uczenia nienadzorowanego. Należy jednak podkreślić, że znane są również prace w zakresie autokoderów z nadzorem (ang. Supervised AutoEncoders SAE), których celem jest rekonstrukcja zarówno danych wejściowych jak i etykiet.

Rolą kodera jest transformacja L wymiarowych danych wejściowych (L - liczba cech, danych surowych, itp.) do E wymiarowych, nowych cech ($E < L$). Transformacja reprezentacji danych musi być tak dokonana, aby dekodery mógł zrekonstruować dane w zmienionej reprezentacji na dane jak najbardziej podobne do reprezentacji źródłowej.

Do celów tego przykładu zapoznamy się z prostym przykładem, w którym będzie używany zbiór danych MNIST Fashion. Na wzór znanego zbioru zeskanowanych cyfr MNIST utworzono zbiór danych MNIST Fashion zawierający 70 000 obrazów (60 000 zestaw treningowy + 10 000 zestaw testowy) o rozdzielczości 28x28 pikseli, każdy przedstawiający zdjęcie odzieży należące do jednej z 10 klas:

0 T-shirt/top, 1 Trouser, 2 Pullover, 3 Dress, 4 Coat, 5 Sandal, 6 Shirt, 7 Sneaker, 8 Bag, 9 Ankle boot

- oryginalny zbiór dostępny jest na stronie: <https://github.com/zalando-research/fashion-mnist> (The MIT License (MIT) Copyright © [2017] Zalando)

W trzecim notatniku zostanie zbudowany nieregularny model sieci neuronowej bez regularyzacji, pracujący na zbiorze danych CIFAR-10 (jest to baza 60 000 kolorowych obrazów o rozmiarze 32x32 piksele wraz z etykietami przypisującymi je do 10 rozłącznych klas (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck), który jest bardziej złożony i trudniejszy do przetworzenia od MNIST – pozwala to przyjrzeć się początkowo problemowi nadmiernego dopasowania. W tym celu zostanie stworzona prosta, w pełni połączona sieć neuronowa przy użyciu modelu sekwencyjnego, gdzie występować będzie warstwa wejściowa, ukryte warstwy gęste z aktywacją ReLU oraz gęsta warstwa

wyjściowa z funkcją aktywacji softmax. Do optymalizacji w początkowej fazie zostanie użyty algorytm optymalizacji SGD (w zadaniach będzie możliwość przeanalizowanie działania innych algorytmów optymalizacji), wraz z funkcją straty entropii krzyżowej, zaś ocenę modelu zrealizuje metryka accuracy. W dalszej części zostaną zaprezentowane sposoby na przeciwdziałanie nadmiernemu dopasowaniu poprzez użycie warstw Dropout, regularyzację wag oraz rozszerzenie zbioru danych treningowych poprzez augmentację.

4. Forma i zawartość sprawozdania

Ocenie podlegać będzie realizacja poszczególnych zadań (fragmenty kodu, analizy i wnioski) w ramach danego ćwiczenia laboratoryjnego. Raport powinien być złożony w formacie PDF i przedstawiać w sposób jasny i przejrzysty wykonane eksperymenty i analizy oraz opatrzone powinny być odpowiednimi wnioskami końcowymi.

Dodatki:

Pliki instruktażowe oraz pliki z zdanymi niezbędne do wykonania ćwiczenia laboratoryjnego:

- 1) *Lab5_ex01.ipynb*
- 2) *lab5_ex02.ipynb*
- 3) *lab5_ex03.ipynb*